Lab 1 — Frame and decompose your own work

Retour à « Labs »

Lab 1 — Frame and decompose your own work

1 September · 3h · lab groups of ~15 · follows Lecture 1 Serves: C1. Continues in the autonomy slot on 3 September.

You work in **your own repository** — an alternance project, a personal project, anything real that you can push to. Not a repository created today.

I have to be able to read it. Prefer a **public** repository: everything you do this week then lands somewhere you can show a recruiter. If it has to stay private, invite me as a viewer before the lab. If you can do neither — an employer's code you cannot open up and cannot share — that repository is not eligible for the course, so choose another one.

The times on the steps below are guidance, not a budget. They add up to more than three hours on purpose — step 1 is the one that overruns, and it overruns for good reasons. Use them to notice you have been grilling for ninety minutes, not to cut a step short because the clock says so.

Step 0 — set yourself up (first 25 minutes, in the room)

Five things, in this order, before step 1. None of them is assessed; all of them are the reason step 1 works.

0.1 No Claude subscription? Set Claude up anyway, on the school backend

The school runs its own LLM backend, and Claude Code talks to it — you point the agent at the school's server instead of Anthropic's and work the lab like everyone else. The setup, the minimal test that proves it is talking, and the answer to "Claude Code asks me to pay" are in **Using Claude Code and OpenCode with the school local LLM backend**, by Thalita Drumond: open it on Moodle. Say so now, not at step 1: it is ten minutes if the prerequisites are in place, and it is not ten minutes you have at 14:20.

0.2 Reset to a clean state

Whatever configuration you have accumulated — MCP servers, skills, plugins, a global `CLAUDE.md` — back it up and take it out. It ships on every turn, none of us can see it, and it is the difference between an agent that behaved strangely and an agent that behaved strangely *because of something you wrote in March*. The prompt that inventories it, moves it to a dated backup and writes its own restore instructions is in Resetting to a clean state.

We do this together, in the first five minutes. The agent does the work; you paste the prompt and read what it reports back. Everybody starts the lab from the same known state, and nobody spends the afternoon debugging a plugin they forgot they installed.

You get it all back at the end of the course, and you will restore less of it than you expect.

0.3 Put the context readout back on

One thing worth putting back before step 1: a context readout. Claude Code does not show you how full your context window is unless you ask it to, and today is a day worth watching that number climb. Showing context in the status line is the setup. Not a lab step, not assessed, and nobody waits for you — but step 5 lands better for having seen the number rise through step 1.

0.4 Choose your repository and your change, and get them validated

Pick a change you actually want in that repository, big enough to have needed decomposing.

The floor: three or more units of work, at least one true blocking edge. A change that decomposes trivially cannot demonstrate this competency. Picking one is a framing failure, and the grid catches it.

Good candidates: a new source ingested end-to-end; a schema change that ripples through transform and reporting; adding a validation layer with a backfill. Bad candidates: renaming things, adding one endpoint, fixing one bug.

Get your choice approved before you continue. Raise a hand. This is the single decision that determines whether the rest of the lab is worth anything, and I would rather redirect ten people in the first twenty minutes than grade ten dead ends on 10 September.

0.5 Install the skills, then run `/setup-matt-pocock-skills`

Install them first — the reset in 0.2 took them out. Steps 1 to 4 each call one of Matt Pocock's skills, and none of the slash commands below exists until you have added them. One command, from inside your repository:

```
npx skills@latest add mattpocock/skills
```

That is the way to do it: it works in Claude Code and in every other harness in the room, it pins nothing to a plugin marketplace, and it puts the skills where the agent will find them. If your harness offers its own plugin installer, ignore it today — we all run the same command so we are all debugging the same setup.

Then set the agent up for this repository, because the skills need to know where your issues live. Run `/setup-matt-pocock-skills`: it asks you three questions about this repository — where the issue tracker is, what its triage labels are called, where the domain docs sit — proposes the answers from your git remote, and writes them into `docs/agents/` once you confirm. Two minutes, and it is the reason `/to-tickets` later files real issues in your tracker instead of printing a list into the chat.

This is also the first thing you have just done that the reset was for: you are adding configuration deliberately, for this repository, and you can say what each piece is doing.

1. Sharpen it by interview (~45 min)

Have the agent interview *you* before it writes anything.

- Claude Code: `/grill-with-docs`. You are in an existing repository, so this is the one to use — it reads what is already there and writes a `CONTEXT.md` and decision records as it goes, rather than leaving the answers in the conversation.
- Any other harness: instruct it to ask you questions in rounds and forbid it from writing code until you say stop. Make it look things up itself; only answer the questions that are genuinely yours to decide.

In French, if English is what is slowing you down. The questions are not in this document — the agent writes them as it goes, so it can write them in French. Add this to the message you run the skill from:

> Mène l'interview en français. Garde en anglais le vocabulaire des agents et du génie logiciel — spec, ticket, context, unit of work, blocking edge, tracer bullet — et écris `CONTEXT.md`, les ADR et les titres de tickets en anglais.

Only the interview changes language. **The words you name things with stay English**, in French sentences, because naming them is what the oral asks of you. If you started in English and you are drowning, say so mid-interview:

> Passe en français pour la suite de l'interview. Garde les termes techniques en anglais.

Do not stop at one round. The useful questions are in rounds two and three, after your first answers have narrowed things.

Done when: the interview has surfaced **at least one question you had not thought about** before it asked. **Write them all down now**, as they come. The oral asks you for one of them — you choose which, so the more you have written the better that choice is.

2. Write the specification (~30 min) — `/to-spec`

What is being built, the decisions taken, and — the part people skip — what is explicitly out of scope.

Recommended: your issue tracker — the spec as an issue, where the tickets in step 3 will hang off it. If your project's conventions say otherwise, a markdown file, a `docs/` entry or a wiki page are all fine. What is not fine is the chat window.

And decide now what happens to it when the work is finished, because it is one line in the spec and it is the kind of thing the oral follow-up asks about. A spec is a projection of the codebase — shorter and denser than the code, which is what makes it useful this week and dangerous next quarter. An agent that finds a stale spec committed as documentation reads it and stops, never reaching the code, which is the only source that cannot lie about itself. So archive it where it was written — a closed issue, or a `docs/` file marked superseded — rather than promoting it to living documentation or deleting it outright. Write that sentence at the bottom of the spec.

Done when: someone who was not in the interview could build the right thing from it, and the ambiguities you named in step 1 are resolved in writing.

3. Split into units of work, with edges (~45 min) — `/to-tickets`

Break the spec into **three or more units of work** — not unit tests; a unit of work is a slice of the change, and what you write down to describe it is its **ticket**. One ticket per unit. Each should be a tracer bullet: a thin vertical slice that runs end to end, not a horizontal layer.

For each, write a ticket: a title, enough description to be executed cold, and **what it is blocked by**.

Then audit every edge you drew. For each one ask: **what actually breaks if I do these the other way round?** If nothing breaks, it is a preference, not an edge. Delete it. You need at least one that survives, and you must be able to say what it breaks.

The test for a unit: could you hand this, and nothing else, to a fresh agent session? If it needs "the thing we discussed earlier", it is not a unit yet.

4. Execute the first unit — in a fresh context (~40 min) — `/implement`

Watch the demo first (5 min). Everyone is about to clear their context, so before you do: a fresh session, inspected before a single word is typed, through `agent-proxy` — a small logging proxy between the agent and the API that writes a Markdown report of everything sent to the model on each turn, ranked by size. The system prompt, every tool schema, the whole history, plus `<tool-use>` and `<system-reminder>`. **Most of what is sent is not your prompt.** That is what fills up, that is what a compaction compresses, and it is why step 5 is a step at all. If the live run misbehaves you get a report generated earlier — the report is the point, not the plumbing. Nothing here is assessed, and you run it yourself on 3 September.

Then: `/clear`, or open a new session. Hand it **one unit** and nothing else — `/implement` with that one ticket.

This is the moment the decomposition is graded by reality. If the agent asks for context that is not in the ticket, the ticket was incomplete — **fix the ticket, not the prompt.** Note what was missing; that note is good evidence for the oral.

Settle model and effort before you start, and then leave them alone. A top-tier model on medium effort. This is not assessed and it is not where your lab should go — if the output disappoints, the leverage is in the ticket you just wrote, not in the settings. One thing worth knowing: raising effort spends context faster, so a high-effort session fits less work than a medium one. The rest is in Model, effort, and what a session costs, which is a read-once document, not a lab step.

If the approach turns out to be wrong — and for someone in this room it will — do not push on through the tickets you no longer believe in. **Tickets are disposable; the spec is editable.** Delete the unimplemented tickets, edit the spec to describe where you now want to end up, and cut new tickets from it. Keep the code you already shipped unless it is actively in the way. Then write down that you did this: a student who rerouted deliberately, and can say what the review told them, is answering the Outstanding-band question about the method's limits without being asked.

5. Record one context decision (~10 min)

Somewhere in this lab you hit a phase boundary. Write down, in two lines: where it was, what you chose (continue / clear / handoff / subagent / compact), and why.

Recommended home: a comment on the issue for the unit of work where the boundary happened. The note is about that unit, so it belongs next to it, and it is then already pushed.

Write the "why" as what you ruled out, in that order. The five moves are ranked: continue and clear cost you nothing, handoff and subagent leave you a document, and compacting is the only one that destroys its own source — so it is the answer you arrive at last, not first. *"I compacted"* on its own is worth nothing in the oral. *"I couldn't continue, and I couldn't clear because the decisions only existed in the conversation"* is the answer, and it also tells you what to write down earlier next time.

If you did not hit one, say so and explain why not — that is also a legitimate answer, and a defensible one is worth more than an invented boundary.

Before you leave

Pushed to your repository:

- ☐ The specification
- ☐ Three or more tickets, one per unit of work, with their blocking edges written down
- ☐ At least one unit executed, in its own context
- ☐ The context decision note
- ☐ The ambiguities the interview surfaced, written down somewhere you will find them on 11 September

Next: the autonomy slot, 3 September (1h30)

Your remaining units are not homework you may or may not get to — they are the second half of this lab, and they have a brief of their own: **Hack and Learn**. Open it on 3 September. It carries the fresh-context work, the proxy run you just watched, and thirty minutes on the vocabulary the oral is conducted in.

Modifié le: mercredi 2 septembre 2026, 19:18